Task 01:

```
#include <iostream>
using namespace std;

// Returns new array of double size:
// first half = original, second half = 2x int*
resizeArray(int inArray[], const int size)
{
    int* newArr = new int[size * 2];
for (int i = 0; i < size; i++)
    {
        newArr[i] = inArray[i];
newArr[i + size] = inArray[i] * 2;
    }
    return newArr;
}

int main()
{     int size;     cout << "Enter size
of the array: ";     cin >> size;

    int* arr = new int[size];     cout
<< "Enter elements of array: ";     for
(int i = 0; i < size; i++)         cin
>> arr[i];     int* res =
resizeArray(arr, size);

    cout << "Resized array: ";     for
(int i = 0; i < size * 2; i++)
    {
        cout << res[i];
if (i < size * 2 - 1)
cout << ", ";
    }     cout <<
endl;

    delete[] arr;
delete[] res;     arr =
res = nullptr;
return 0;
}
```

Task 02:

```
#include <iostream>
using namespace std;

// true if all entries outside main diagonal are 0
bool isDiagonal(int** m, int n)
{
    for (int i = 0; i < n; i++)
for (int j = 0; j < n; j++)
if (i != j && m[i][j] != 0)
return false;     return true; }
```

```
int main()
{     int n;     cout << "Enter
matrix size: ";     cin >> n;

    // allocate n x n matrix
int** m = new int*[n];     for
(int i = 0; i < n; i++)
m[i] = new int[n];

    cout << "Enter matrix values:\n";
for (int i = 0; i < n; i++)
for (int j = 0; j < n; j++)
cin >> m[i][j];

    if (isDiagonal(m, n))
cout << "Diagonal matrix\n";
else
        cout << "Not a diagonal matrix\n";

    // free memory     for (int
i = 0; i < n; i++)
delete[] m[i];     delete[] m;
m = nullptr;     return 0;
}
```

Task 03:

```
#include <iostream>
#include <cstring>
using namespace std;

// copies first 'count' chars of target char*
left(const char* target, const int count)
{
    char* sub = new char[count + 1];
for (int i = 0; i < count; i++)
sub[i] = target[i];     sub[count] =
'\0';     return sub; }

// frees the dynamic memory
void Destructor(char* p)
{     delete[]
p; }

int main()
{     char
str[30];     int
count;

    cout << "Enter a string: ";     cin.getline(str,
30);     if (cin.fail())          // input longer
than 29
    {         cin.clear();
cin.ignore(1000, '\n');     }

    cout << "Enter count: ";
cin >> count;
```

```
    if (count < 0 || count > (int)strlen(str))
cout << "Count is out of range!\n";      else
{
        char* sub = left(str, count);         cout
<< "Left substring: " << sub << endl;
        Destructor(sub);
sub = nullptr;
    }
return 0;
}
```

Task 04:

```
#include <iostream>
using namespace std;

int findMin(int inArray[], const int size)
{
    int min = inArray[0];     for
(int i = 1; i < size; i++)
if (inArray[i] < min)
min = inArray[i];     return min;
}

int findMax(int inArray[], const int size)
{
    int max = inArray[0];     for
(int i = 1; i < size; i++)
if (inArray[i] > max)
max = inArray[i];     return max;
}

int computeAverage(int inArray[], const int size)
{     int sum = 0;     for (int i
= 0; i < size; i++)         sum +=
inArray[i];     return sum / size;
}

int computeMedian(int inArray[], const int size)
{
    // sort a copy (bubble sort)     int* t =
new int[size];     for (int i = 0; i < size;
i++)         t[i] = inArray[i];     for (int i
= 0; i < size - 1; i++)         for (int j =
0; j < size - i - 1; j++)             if (t[j]
> t[j + 1])
            {
int tmp = t[j];
t[j] = t[j + 1];
t[j + 1] = tmp;             }

    int med;     if (size
% 2 == 1)         med =
t[size / 2];     else
        med = (t[size / 2 - 1] + t[size / 2]) / 2;

    delete[] t;
return med; }

int main()
```

```
{     int
n;
cout <<
"Number of
students:
";     cin
>> n;
while (n
<= 0)
    {
        cout << "Enter a positive number: ";
cin >> n;     }

    int* marks = new int[n];
for (int i = 0; i < n; i++)
    {
        cout << "Marks of student " << i + 1 << ": ";
cin >> marks[i];         while (marks[i] < 0)   // no
negatives
        {
            cout << "Invalid! Re-enter: ";
cin >> marks[i];
        }
    }

    cout << "Maximum: " << findMax(marks, n) << endl;
cout << "Minimum: " << findMin(marks, n) << endl;     cout
<< "Average: " << computeAverage(marks, n) << endl;     cout
<< "Median : " << computeMedian(marks, n) << endl;

    delete[] marks;
marks = nullptr;
return 0;
}
```